

**DESIGN AND IMPLEMENTATION OF A WEB-BASED INTERACTIVE E-
LEARNING PLATFORM FOR STRUCTURED PYTHON PROGRAMMING
(A CASE STUDY OF MAYOR4CODE)**

BY

Giwa Mayowa Bopoola

CSC/22/204

DEPARTMENT OF COMPUTER SCIENCE

SCHOOL OF COMPUTING

OLUSEGUN AGAGU UNIVERSITY OF SCIENCE AND TECHNOLOGY,
OKITIPUPA

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
THE DEGREE OF BACHELOR OF SCIENCE (B.Sc.) IN COMPUTER SCIENCE

August, 2026

CERTIFICATION

This is to certify that this project titled “Design and Implementation of a Web-Based Interactive E-Learning Platform for Structured Python Programming (A Case Study of mayor4code)” was carried out by Giwa Mayowa Bopoola with matriculation number CSC/22/204 in the Department of Computer Science, School of Computing, Olusegun Agagu University of Science and Technology, Okitipupa, in partial fulfilment of the requirements for the award of the degree of Bachelor of Science (B.Sc.) in Computer Science.

_____	_____
Dr. Adeolu Obamehinti	Dr. Ajoke Gbadamosi
Project Supervisor	Head of Department
Date: _____	Date: _____

DECLARATION

I hereby declare that this project is my original work and has not been submitted, either in whole or in part, for any other degree or qualification in this or any other institution. All sources of information and materials used have been duly acknowledged by means of complete references.

Giwa Mayowa Bopoola

CSC/22/204

Date: _____

DEDICATION

This project is dedicated to Almighty God, and to my family, whose support and encouragement made this work possible.

ACKNOWLEDGEMENTS

I express my sincere gratitude to my project supervisor, Dr. Adeolu Obamehinti, for the guidance, patience, and constructive criticism that shaped this work. I am grateful to the Head and entire staff of the Department of Computer Science, Olusegun Agagu University of Science and Technology, Okitipupa, for the knowledge imparted throughout my programme. I also thank my family and friends for their unwavering support and encouragement. Above all, I give thanks to Almighty God for His grace and strength.

ABSTRACT

Learning to program is widely regarded as difficult for beginners, largely because traditional methods separate instructional content from practical coding and provide little immediate feedback. This project presents the design and implementation of mayor4code, a web-based interactive e-learning platform for structured Python programming. The system was developed using the Agile approach within the Software Development Life Cycle and adopts a three-tier architecture comprising presentation, application, and database layers. It was implemented using the Django web framework, the Python programming language, HTML, CSS, and JavaScript, with SQLite used during development and PostgreSQL in production. The platform delivers twelve sequential Python lessons with locked progression, in which each lesson is unlocked only after the learner passes the preceding quiz at a minimum of sixty percent. It further provides multiple-choice quizzes with instant automated scoring, an in-browser Python playground that executes learner code in an isolated subprocess, progress tracking, automatically issued completion certificates bearing unique verification codes, and a leaderboard that ranks learners by performance. The platform was tested through unit, functional, and usability testing, and the results showed that it correctly enforced lesson progression, scored assessments accurately, executed code safely, and issued valid certificates. The study concludes that integrating structured lessons, practical coding, automated assessment, and gamification within a single secure web application improves the accessibility and effectiveness of introductory programming education. It is recommended that the platform be extended with additional courses and adaptive learning features in future work.

TABLE OF CONTENTS

CHAPTER ONE INTRODUCTION	1
1.1 Background of the Study.....	1
1.2 Research Justification/Motivation.....	2
1.3 Problem Statement	2
1.4 Aim and Objectives of the Study	3
Aim.....	3
Objectives.....	3
1.5 Scope of the Study	3
1.6 Significance of the Study	4
1.6.1 Educational Impact.....	4
1.6.2 Technological Impact.....	4
1.6.3 Economical Impact.....	4
1.7 Definition of Terms.....	5
1.8 Organization of the Study	5
CHAPTER TWO LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Concept of E-Learning Systems.....	7
2.3 Interactive Programming Learning Platforms.....	7
2.4 Learning Management and Progress Tracking	8
2.5 Gamification in Learning	8
2.6 Technologies Used in Web Development.....	8
2.7 Related Works and Existing Systems	9
2.7.1 Codecademy Interactive Coding Platform	9
2.7.2 freeCodeCamp Learning Platform	10
2.7.3 SoloLearn Mobile Learning Application	10
2.7.4 HackerRank Skill Assessment Platform	11

2.7.5 W3Schools Online Tutorials	11
2.7.6 Summary of Related Works	12
2.8 Research Gap	12
2.9 Summary of Literature Review	13
CHAPTER THREE METHODOLOGY	14
3.1 Introduction	14
3.2 Project Design and Approach.....	14
3.3 Design Consideration	14
Security	14
Usability	15
Accessibility.....	15
Scalability.....	15
Reliability.....	15
Performance	15
3.4 System Architecture	15
3.5 Components of System Architecture	16
User Interface Module.....	16
Authentication Module.....	16
Lesson and Progression Module	16
Quiz and Assessment Module.....	16
Code Playground Module	17
Certification and Leaderboard Module	17
Administration Module	17
3.6 Database Design.....	17
3.7 Flowchart of the System.....	18
CHAPTER FOUR SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION	19

4.1 Introduction	19
4.2 System Implementation.....	19
4.3 Development Tools and Environment	19
4.4 System Interfaces	20
4.4.1 Registration and Login Interfaces	20
4.4.2 Dashboard Interface	21
4.4.3 Lesson and Quiz Interfaces	22
4.4.4 Python Playground Interface.....	24
4.4.5 Certificate and Leaderboard Interfaces	25
4.4.6 Administration Interface	26
4.5 System Testing	27
4.5.1 Unit Testing.....	27
4.5.2 Functional Testing.....	27
4.5.3 Usability Testing	27
4.6 Discussion of Results	28
CHAPTER FIVE SUMMARY, CONCLUSION AND RECOMMENDATIONS...	29
5.1 Summary of Findings.....	29
5.2 Conclusion	29
5.3 Recommendations	29
5.4 Suggestions for Future Research.....	30
REFERENCES.....	31

LIST OF FIGURES

Figure 3.1: System Architecture of the mayor4code Platform

Figure 3.2: System Flowchart of the mayor4code Platform

Figure 4.1: Registration Interface

Figure 4.2: Login Interface

Figure 4.3: Learner Dashboard

Figure 4.4: Lesson Interface

Figure 4.5: Quiz Interface

Figure 4.6: Python Playground Interface

Figure 4.7: Completion Certificate

Figure 4.8: Leaderboard Interface

Figure 4.9: Administration Interface

LIST OF TABLES

Table 3.1: Summary of Principal Database Entities

Table 4.1: Software Development Tools

Table 4.2: Summary of Test Cases and Results

LIST OF ABBREVIATIONS

API — Application Programming Interface

CSS — Cascading Style Sheets

DBMS — Database Management System

HTML — HyperText Markup Language

HTTP — HyperText Transfer Protocol

LMS — Learning Management System

ORM — Object-Relational Mapper

SDLC — Software Development Life Cycle

SQL — Structured Query Language

UI — User Interface

CHAPTER ONE INTRODUCTION

1.1 Background of the Study

The rapid advancement of information and communication technology has transformed the way knowledge is delivered, acquired, and assessed across all levels of education. Electronic learning (e-learning) platforms have emerged as powerful tools that extend learning beyond the physical classroom, enabling learners to study at their own pace, from any location, using internet-connected devices. In the field of computer programming, where practical skill and continuous practice are essential, interactive e-learning platforms have proven particularly effective because they combine instructional content with hands-on coding activities.

Learning to program is widely regarded as challenging for beginners. Traditional methods of teaching programming, which rely heavily on lectures and textbooks, often fail to provide the immediate feedback and practical engagement that novice programmers require. According to Pressman and Maxim (2020), interactive software systems that provide timely feedback significantly improve user engagement and learning outcomes. Interactive coding platforms address this gap by allowing learners to write, execute, and test code directly within the browser while receiving instant feedback on their progress.

A structured e-learning platform organises instructional content into sequential lessons and assessments, guiding learners progressively from basic to advanced concepts. Such platforms typically incorporate features such as user authentication, progress tracking, automated assessment, and certification. Sommerville (2016) noted that web-based applications offer flexibility, scalability, and ease of maintenance, making them well suited for modern educational systems that must serve many concurrent users across diverse devices.

This study presents the design and implementation of mayor4code, a web-based interactive e-learning platform for structured Python programming. The platform delivers twelve sequential Python lessons with locked progression, automated multiple-choice quizzes, an in-browser Python playground for practical coding, automatically issued completion certificates, and a leaderboard that fosters healthy competition among learners. By integrating structured content delivery with

interactive practice and automated assessment, the platform seeks to make the process of learning Python more engaging, accessible, and effective for beginners.

1.2 Research Justification/Motivation

The motivation for this research stems from the increasing demand for accessible and practical programming education. Python has become one of the most widely adopted programming languages in academia and industry due to its simple syntax and broad application in areas such as data science, artificial intelligence, and web development. However, many beginners struggle to acquire programming skills through conventional learning methods that lack interactivity and immediate feedback.

Existing programming education is often fragmented, requiring learners to switch between separate tools for reading lessons, writing code, and taking assessments. This fragmentation increases cognitive load and discourages consistent practice. There is therefore a need for an integrated platform that combines structured lessons, practical coding, and assessment within a single, coherent environment.

Furthermore, the principle of locked progression, in which each lesson is unlocked only after the learner demonstrates mastery of the preceding one, encourages disciplined and thorough learning. Coupled with gamification elements such as certificates and a leaderboard, this approach is expected to improve learner motivation and retention. These considerations motivated the development of mayor4code as a self-contained, interactive, and structured platform for learning Python programming.

1.3 Problem Statement

Despite the growing availability of online learning resources, beginners learning to program continue to face significant challenges. Many freely available resources present content in an unstructured manner, leaving learners uncertain about the order in which topics should be studied. This lack of structure frequently results in knowledge gaps and discourages learners from completing their studies.

A second challenge is the separation between theory and practice. Learners often read programming concepts without an immediate opportunity to apply them, and configuring a local programming environment can be difficult and discouraging for

beginners. The absence of instant, automated feedback further limits learners' ability to identify and correct mistakes promptly.

In addition, many learning resources provide no mechanism for tracking progress, enforcing mastery before advancement, or recognising achievement through certification. The absence of these features reduces motivation and makes it difficult for learners to measure their advancement objectively.

This study addresses these challenges through the development of mayor4code, a web-based interactive e-learning platform that provides structured lesson progression, an integrated code execution environment, automated assessment with instant scoring, progress tracking, and automated certification, all within a single secure and user-friendly system.

1.4 Aim and Objectives of the Study

Aim

The aim of this study is to design and implement a web-based interactive e-learning platform for structured Python programming that integrates lesson delivery, practical coding, and automated assessment within a single system.

Objectives

The specific objectives of the study are to:

1. design a web-based platform that delivers structured Python lessons with locked progression;
2. develop a secure user authentication and authorisation system for learners;
3. implement an interactive in-browser Python playground for practical code execution;
4. develop an automated quiz and instant-scoring mechanism to assess learner mastery;
5. implement progress tracking, automated certification, and a leaderboard to enhance learner motivation;
6. evaluate the functionality and usability of the developed platform.

1.5 Scope of the Study

This project focuses on the design and implementation of a web-based interactive e-learning platform for learning the fundamentals of Python programming. The

platform provides functionalities including user registration and authentication, sequential delivery of twelve Python lessons, locked lesson progression based on quiz performance, multiple-choice quizzes with instant scoring, an in-browser Python playground, progress tracking, automated completion certificates, and a leaderboard.

The system primarily serves learners (students) and an administrator who manages lessons, quizzes, questions, and user progress. The study is limited to the teaching and assessment of introductory Python programming and does not extend to other programming languages, live video instruction, or peer-to-peer collaboration features.

1.6 Significance of the Study

The development of the mayor4code e-learning platform provides several benefits to learners, educators, and educational institutions.

1.6.1 Educational Impact

The platform promotes structured and self-paced learning of Python programming. By enforcing mastery before progression and providing instant feedback through automated quizzes and an interactive playground, it enhances comprehension, retention, and practical skill acquisition among beginners.

1.6.2 Technological Impact

The project demonstrates the practical application of modern web technologies, database management systems, and secure code-execution techniques in solving a real-world educational problem. It contributes to the digital transformation of programming education by providing an integrated and automated learning environment.

1.6.3 Economical Impact

The platform reduces the cost associated with traditional programming instruction, such as physical infrastructure, printed materials, and one-on-one tutoring. As a web-based solution, it enables a single deployment to serve many learners simultaneously, thereby improving cost efficiency and accessibility.

1.7 Definition of Terms

E-Learning: The delivery of learning and training through digital resources accessed over the internet or a computer network.

Interactive Platform: A software system that responds to user actions in real time, enabling active participation rather than passive consumption of content.

Locked Progression: A learning mechanism in which access to subsequent lessons is granted only after the learner satisfies a defined requirement, such as passing the preceding quiz.

Playground: An in-browser environment in which learners can write and execute program code and view the output instantly.

Quiz: A set of multiple-choice questions used to assess a learner's understanding of a lesson.

Certificate: A digitally issued document, bearing a unique verification code, that recognises a learner's completion of the course.

Authentication: The process of verifying the identity of a user before granting access to the system.

Leaderboard: A ranked display of learners based on their performance, used to encourage motivation and engagement.

1.8 Organization of the Study

This project report is organised into five chapters.

Chapter One presents the introduction, background of the study, motivation, problem statement, aim and objectives, scope, significance, definition of terms, and organisation of the study.

Chapter Two reviews relevant literature related to e-learning systems, interactive programming education, learning management systems, gamification, and existing platforms.

Chapter Three describes the methodology adopted for the design and development of the proposed system, including the design approach, system architecture, and implementation tools.

Chapter Four presents the implementation details, system interfaces, testing procedures, results, and discussion of findings.

Chapter Five provides the summary of the study, conclusion, recommendations, and suggestions for future research.

CHAPTER TWO LITERATURE REVIEW

2.1 Introduction

This chapter reviews relevant literature related to e-learning systems, interactive programming education, learning management systems, gamification in learning, web development technologies, and existing programming-learning platforms. The review provides an understanding of previous work and identifies the gaps that justify the development of the proposed mayor4code platform.

2.2 Concept of E-Learning Systems

E-learning refers to the use of electronic technologies and the internet to deliver educational content and facilitate learning outside the traditional classroom. E-learning systems enable learners to access instructional materials, complete assessments, and monitor their progress from any location using internet-connected devices. According to Sommerville (2016), web-based systems support distributed access to resources, thereby improving operational efficiency and user convenience.

E-learning has been widely adopted in education because it offers flexibility, scalability, and cost effectiveness. Garrison (2017) observed that e-learning supports self-paced, learner-centred education that can reach large numbers of learners simultaneously, while Anderson (2008) noted that well-designed online learning environments can be as effective as traditional instruction. In the context of programming education, e-learning systems are particularly valuable because they can integrate practical coding activities directly into the learning environment.

2.3 Interactive Programming Learning Platforms

Interactive programming learning platforms combine instructional content with hands-on coding activities, enabling learners to write and execute code while studying. Unlike static resources such as textbooks and video tutorials, these platforms provide immediate feedback that helps learners identify and correct errors promptly.

The provision of instant feedback is grounded in established learning theory. Prompt reinforcement strengthens understanding and encourages continued practice, which is essential in skill-based disciplines such as programming. Interactive platforms also

reduce the barrier posed by local environment setup, since code is executed within the browser, allowing beginners to focus on learning rather than configuration.

2.4 Learning Management and Progress Tracking

A learning management system (LMS) is a software application used to plan, deliver, and track educational content and learner performance. Core LMS functions include user management, content delivery, assessment, progress tracking, and reporting. Progress tracking enables learners to monitor their advancement and helps maintain motivation by providing a clear sense of accomplishment.

Locked or sequential progression is a design strategy in which learners must satisfy defined criteria, such as passing an assessment, before advancing to the next unit. This approach enforces mastery of prerequisite concepts and reduces knowledge gaps. Automated assessment, in which quizzes are scored instantly by the system, further supports timely feedback and reduces the administrative burden associated with manual grading.

2.5 Gamification in Learning

Gamification refers to the application of game design elements, such as points, badges, certificates, and leaderboards, in non-game contexts to increase engagement and motivation. Deterding, Dixon, Khaled, and Nacke (2011) defined gamification as the use of game-design elements in non-game contexts, and in education it has been shown to enhance learner participation, persistence, and satisfaction. Hamari, Koivisto, and Sarsa (2014), in a review of empirical studies, found that gamification generally produces positive effects on engagement, though the outcomes depend on context and implementation.

Elements such as completion certificates provide learners with tangible recognition of achievement, while leaderboards introduce an element of healthy competition that can motivate continued effort. When applied appropriately, these mechanisms complement structured learning by reinforcing desired behaviours and sustaining learner interest throughout the course.

2.6 Technologies Used in Web Development

Web development involves the use of various technologies for designing and implementing web applications. The front-end, responsible for the user interface, is

commonly built using HTML, CSS, and JavaScript. Duckett (2014) explained that HTML provides the structure of web pages, CSS controls presentation and styling, and JavaScript enables interactive functionality; together these form the foundation of modern web development.

The back-end handles server-side operations such as authentication, business logic, and database interactions. Popular back-end technologies include Python, PHP, Node.js, and Java. The proposed system employs the Django web framework, a high-level Python framework that promotes rapid development and clean, pragmatic design, and which incorporates built-in security features and an object-relational mapper for database management (Django Software Foundation, 2024). Python itself is a general-purpose, high-level language noted for its readable syntax (Python Software Foundation, 2024).

Data in web applications are managed by a database management system (DBMS). Elmasri and Navathe (2017) noted that database systems ensure data consistency, integrity, security, and efficient retrieval of information. The proposed system uses SQLite during development and PostgreSQL in production, both of which are reliable relational database systems well supported by the Django framework.

2.7 Related Works and Existing Systems

2.7.1 Codecademy Interactive Coding Platform

Approach

Codecademy is a commercial online platform that teaches programming through interactive, in-browser lessons. Learners read short instructional segments and immediately apply concepts in an integrated code editor that validates their solutions.

Architectural Framework

The platform combines a web-based user interface, an in-browser code editor and execution environment, an automated exercise-validation engine, and a centralised database that stores user accounts and progress.

Strengths

- Provides highly interactive, hands-on learning.
- Offers structured learning paths across many languages.
- Delivers immediate feedback on exercises.

Weaknesses

- Most advanced content is restricted behind a paid subscription.
- Certification is limited in the free tier.
- Requires continuous internet connectivity.

2.7.2 freeCodeCamp Learning Platform

Approach

freeCodeCamp is a free, non-profit platform that teaches web development and programming through a sequence of interactive challenges and projects, leading to free certifications.

Architectural Framework

It consists of a web interface, an in-browser challenge editor, an automated test runner that validates learner solutions, and a database that tracks completion and issues certifications.

Strengths

- Completely free and open-source.
- Offers project-based learning and free certification.
- Maintains a large supportive community.

Weaknesses

- Focuses primarily on web development rather than general programming.
- Self-directed structure may overwhelm absolute beginners.
- Limited personalised guidance.

2.7.3 SoloLearn Mobile Learning Application

Approach

SoloLearn teaches programming through bite-sized lessons and quizzes, with a strong emphasis on mobile learning and community interaction.

Architectural Framework

The system integrates a mobile and web interface, a lesson and quiz engine, a code playground, and social features backed by a centralised database.

Strengths

- Bite-sized lessons suited to beginners.
- Includes gamification through points and streaks.
- Provides a mobile-friendly experience.

Weaknesses

- Content depth is limited for advanced learners.
- Heavy reliance on community-generated content quality.
- Advertisements in the free version.

2.7.4 HackerRank Skill Assessment Platform

Approach

HackerRank provides coding challenges and skill assessments primarily for practising programming and preparing for technical interviews.

Architectural Framework

It comprises a web interface, an online code editor supporting multiple languages, an automated judging engine, and a database for storing submissions and rankings.

Strengths

- Extensive library of practice problems.
- Automated evaluation with instant results.
- Supports many programming languages.

Weaknesses

- Oriented towards practice and assessment rather than structured teaching.
- Challenging for complete beginners.
- Limited explanatory lesson content.

2.7.5 W3Schools Online Tutorials

Approach

W3Schools provides free reference tutorials for web and programming technologies, featuring simple explanations and an in-browser "Try it Yourself" editor.

Architectural Framework

The platform consists of static tutorial pages, an embedded code editor for experimentation, and optional quiz and certification services.

Strengths

- Free and easy to navigate.
- Includes editable code examples.
- Broad coverage of web technologies.

Weaknesses

- Reference style lacks enforced structured progression.
- No mastery-based locking of content.
- Limited automated progress tracking.

2.7.6 Summary of Related Works

The reviewed platforms demonstrate the effectiveness of interactive, browser-based programming education and the value of automated feedback, gamification, and certification. However, they also reveal recurring limitations, including paywalls, insufficient structured progression, weak enforcement of mastery before advancement, and a frequent separation between teaching and practice. These observations inform the design of the proposed system.

2.8 Research Gap

Based on the reviewed literature and existing systems, the following research gaps were identified:

7. Limited Free and Structured Progression: Many effective platforms restrict structured learning paths behind paid subscriptions, while free alternatives often lack enforced sequential progression.
8. Weak Enforcement of Mastery: Several systems allow learners to advance without demonstrating understanding of prerequisite concepts, leading to knowledge gaps.
9. Separation of Theory and Practice: Some resources present lessons and coding environments separately, increasing cognitive load for beginners.
10. Inconsistent Certification: Free platforms frequently provide limited or no verifiable certification of completion.

11. Limited Integration in a Single System: Few lightweight platforms combine structured lessons, an integrated code playground, automated assessment, progress tracking, certification, and a leaderboard within one coherent environment.

2.9 Summary of Literature Review

This chapter reviewed literature related to e-learning systems, interactive programming education, learning management and progress tracking, gamification, web development technologies, and existing programming-learning platforms. The review revealed that interactive, browser-based learning with automated feedback significantly improves programming education, and that gamification and certification enhance motivation. However, limitations such as paywalls, weak mastery enforcement, and the fragmentation of learning tools remain evident. These gaps justify the development of the proposed mayor4code platform, which integrates structured lessons, locked progression, an interactive playground, automated assessment, certification, and a leaderboard within a single secure web application.

CHAPTER THREE METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the design and implementation of the mayor4code web-based interactive e-learning platform. It describes the design approach, the considerations made during system development, the system architecture and its components, the database design, and the operational flow of the system. The methodology provides a structured framework for developing a reliable, secure, and user-friendly platform for structured Python learning.

3.2 Project Design and Approach

The development of the mayor4code platform followed the Software Development Life Cycle (SDLC) using the Agile development approach. The Agile methodology was adopted because it supports iterative development, continuous improvement, flexibility, and regular testing throughout the development process. Features were developed incrementally and refined based on repeated testing.

The system was designed as a web-based application accessible through a web browser. It defines two principal roles: the Learner (student), who registers, studies lessons, takes quizzes, uses the code playground, and earns certificates; and the Administrator, who manages lessons, quizzes, questions, and user progress through a dedicated administrative interface.

The system adopts a three-tier architecture consisting of the presentation layer, the application layer, and the database layer. This separation promotes scalability, maintainability, and a clear division of responsibilities within the system.

3.3 Design Consideration

Several factors were considered during the design of the proposed system to ensure effectiveness and usability.

Security

The system was designed with secure user authentication and authorisation. Passwords are hashed, sessions are protected, and requests are guarded against cross-site request forgery. The code playground executes learner code in an isolated subprocess with an enforced time limit to protect the server.

Usability

A clean, responsive, and intuitive interface was designed so that learners can navigate the platform easily. A dark and light mode toggle was provided to improve reading comfort, with the preference persisting across sessions.

Accessibility

The platform is web-based and can be accessed through computers, tablets, and mobile devices connected to the internet, ensuring learning is not restricted to a single location or device.

Scalability

The system was designed to accommodate an increasing number of learners and content. The use of the Django framework and a relational database supports growth without significant redesign, and PostgreSQL is employed in the production environment for robustness.

Reliability

The database schema and application logic were designed to ensure data integrity and consistent performance, so that learner progress and assessment results are recorded accurately.

Performance

Static assets are served efficiently, and the locked-progression logic and quiz scoring were implemented to respond promptly, providing learners with a responsive experience.

3.4 System Architecture

The proposed mayor4code platform adopts a three-tier architecture model consisting of the Presentation Layer, the Application Layer, and the Database Layer.

The Presentation Layer is the interface through which users interact with the system using a web browser. It is built with HTML, CSS, and JavaScript and provides pages for registration, login, the dashboard, lessons, quizzes, the code playground, the leaderboard, and certificates.

The Application Layer contains the business logic, implemented using the Django framework in Python. It processes user requests, manages authentication and

authorisation, enforces locked lesson progression, scores quizzes, executes playground code in a controlled subprocess, issues certificates, and computes leaderboard rankings.

The Database Layer stores all persistent data, including user accounts, lesson content, quizzes and questions, learner progress records, and issued certificates. SQLite is used during development and PostgreSQL in production. Interaction among these three layers ensures secure, efficient communication between users and the system.

Figure 3.1: System Architecture of the mayor4code Platform

3.5 Components of System Architecture

The architecture of the proposed system consists of the following major components:

User Interface Module

This component provides the graphical interface through which learners and the administrator interact with the system, including forms, dashboards, lesson pages, quiz pages, and the code playground.

Authentication Module

This module handles user registration, login, logout, and password reset. It verifies credentials and controls access based on the user's role and authentication status.

Lesson and Progression Module

This module delivers the twelve structured Python lessons and enforces locked progression, unlocking each lesson only after the learner passes the preceding lesson's quiz at or above the defined pass mark of sixty percent.

Quiz and Assessment Module

This module presents multiple-choice quizzes one question at a time, scores responses automatically, and records the results, providing learners with instant feedback on their performance.

Code Playground Module

This module provides an in-browser environment in which learners can write and execute Python code. Submitted code is run in an isolated subprocess with an enforced execution time limit and safeguards against destructive operations.

Certification and Leaderboard Module

This module automatically issues a completion certificate bearing a unique verification code when a learner completes all lessons, and it ranks learners on a leaderboard according to their average performance.

Administration Module

This module allows the administrator to manage lessons, quizzes, questions, users, and progress records, and to monitor overall system activity.

3.6 Database Design

The database was designed to store all persistent information required by the system in a structured and consistent manner. The principal entities and their relationships are summarised as follows:

- User: stores learner account details, including username, email, and hashed password.
- Lesson: stores each lesson's title, content, and order in the sequence.
- Quiz: associated with a lesson and containing its set of questions.
- Question: stores the question text, options, and the correct answer for a quiz.
- UserLessonProgress: records each learner's status and score for each lesson, supporting locked progression.
- Certificate: stores issued certificates, each with a unique verification code linked to the learner.

A user may have many progress records and one certificate; a lesson has one associated quiz; and a quiz has many questions. These relationships preserve data integrity and support efficient retrieval of learner progress and assessment results.

Table 3.1: Summary of Principal Database Entities

3.7 Flowchart of the System

The system begins when a user accesses the platform through a web browser. New users register for an account, while returning users log in with their credentials. Upon successful authentication, the learner is directed to a dashboard that displays overall progress and the available lessons.

The learner studies the first lesson and then attempts its quiz. The system scores the quiz automatically; if the learner attains at least the pass mark, the next lesson is unlocked, otherwise the learner may review the lesson and re-attempt the quiz. At any time, the learner may use the code playground to write and execute Python code and view the output instantly.

When a learner successfully completes all lessons, the system automatically issues a completion certificate with a unique verification code, and the learner's performance is reflected on the leaderboard. The administrator, through a separate interface, manages lessons, quizzes, questions, users, and progress records. The process continues until the user completes the intended operation and logs out, which terminates the active session and ensures system security.

Figure 3.2: System Flowchart of the mayor4code Platform

CHAPTER FOUR SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the implementation of the mayor4code web-based interactive e-learning platform, the development tools and environment used, the system interfaces, the testing procedures adopted, and a discussion of the results obtained. The discussion relates the findings to the objectives set out in Chapter One and to the gaps identified in the literature reviewed in Chapter Two.

4.2 System Implementation

The system was implemented as a web application following the three-tier architecture and design described in Chapter Three. The back-end was developed using the Django web framework in Python, which provided the object-relational mapper, authentication facilities, and request-handling logic. The front-end was implemented using HTML, CSS, and JavaScript to deliver a responsive and interactive user interface. SQLite served as the database during development, while PostgreSQL was configured for the production environment.

The application logic was organised into modular components corresponding to the authentication, lesson and progression, quiz and assessment, code playground, certification and leaderboard, and administration functions. Static assets were served efficiently in production using the WhiteNoise library, and application configuration was managed through environment variables to separate settings from source code and protect sensitive credentials.

4.3 Development Tools and Environment

The tools and technologies used in the development of the system are summarised in Table 4.1.

Table 4.1: Software Development Tools

- Python — Core back-end programming language.
- Django — High-level Python web framework for the application layer.
- SQLite / PostgreSQL — Relational database management systems (development / production).

- HTML, CSS, JavaScript — Front-end structure, styling, and interactivity.
- WhiteNoise — Efficient static-file serving in production.
- Git — Version control for source-code management.
- Visual Studio Code — Source-code editor used for development.
- Web browser — Client platform for accessing and testing the application.

4.4 System Interfaces

This section presents the major interfaces of the implemented system and describes the function of each.

4.4.1 Registration and Login Interfaces

The registration interface allows a new learner to create an account by supplying a username, email address, and password, with a show/hide password toggle for convenience. The login interface authenticates returning users before granting access to the protected areas of the platform, as shown in Figure 4.1 and Figure 4.2.

The screenshot displays the registration interface of the mayor4code platform. The interface is divided into a purple sidebar and a white main content area. The sidebar contains the text "Start learning Python today." and statistics: 5 Lessons, ∞ Attempts, Leaderboard, and Certificate. The main content area is titled "Create your account" and includes a "Continue with Google" button, a "Username" field, a "Matric Number" field, a "Password" field with a show/hide toggle, and a "Confirm Password" field. A green "Create Account" button is at the bottom. The footer contains navigation links for Platform, Account, and Learn, along with copyright information.

Figure 4.1: Registration Interface of the mayor4code Platform

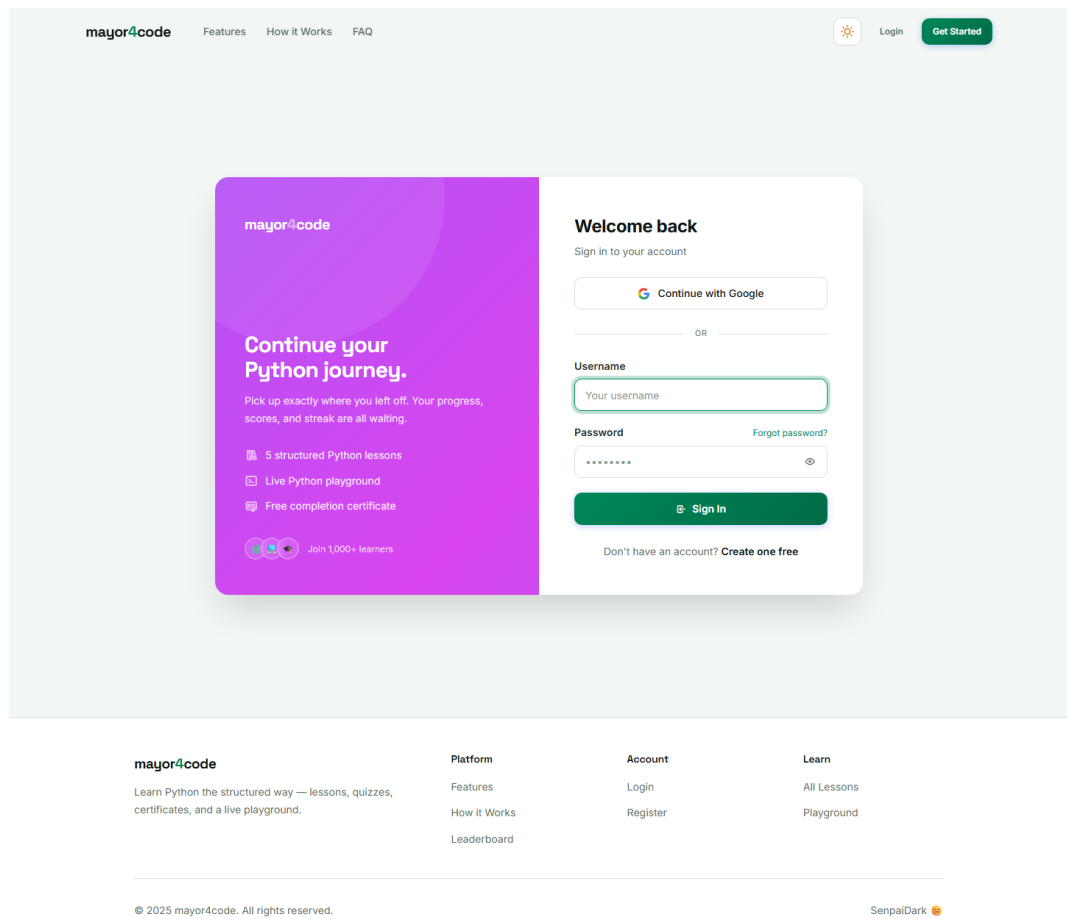


Figure 4.2: Login Interface of the mayor4code Platform

4.4.2 Dashboard Interface

Upon successful login, the learner is presented with a dashboard, shown in Figure 4.3, that displays an overall progress bar and the list of lessons, indicating which lessons are unlocked, completed, or still locked.

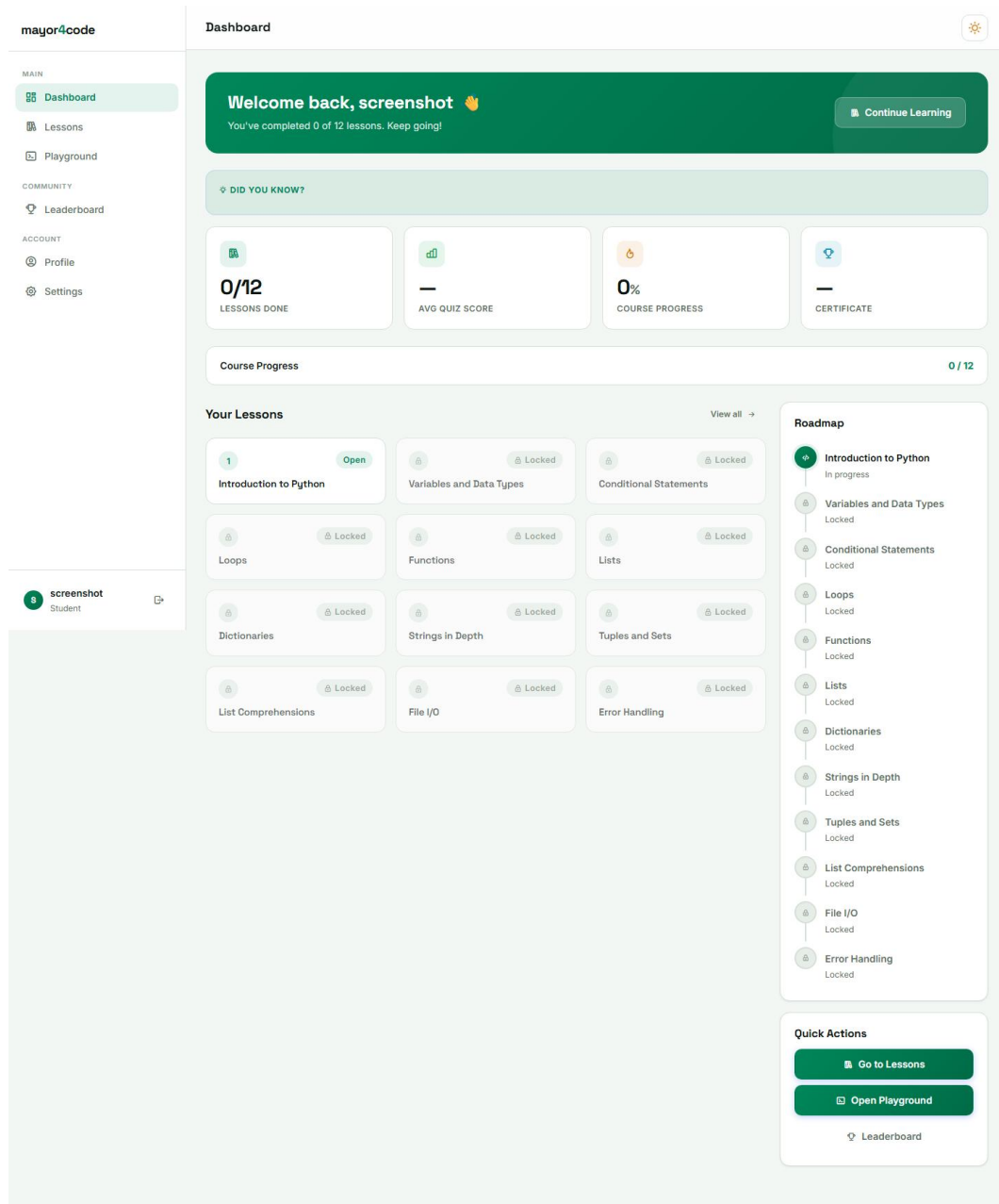


Figure 4.3: Learner Dashboard of the mayor4code Platform

4.4.3 Lesson and Quiz Interfaces

The lesson interface, shown in Figure 4.4, presents lesson content with previous and next navigation. On completing a lesson, the learner attempts its quiz through the quiz interface, shown in Figure 4.5, which presents one multiple-choice question at a time and displays the score instantly upon submission.

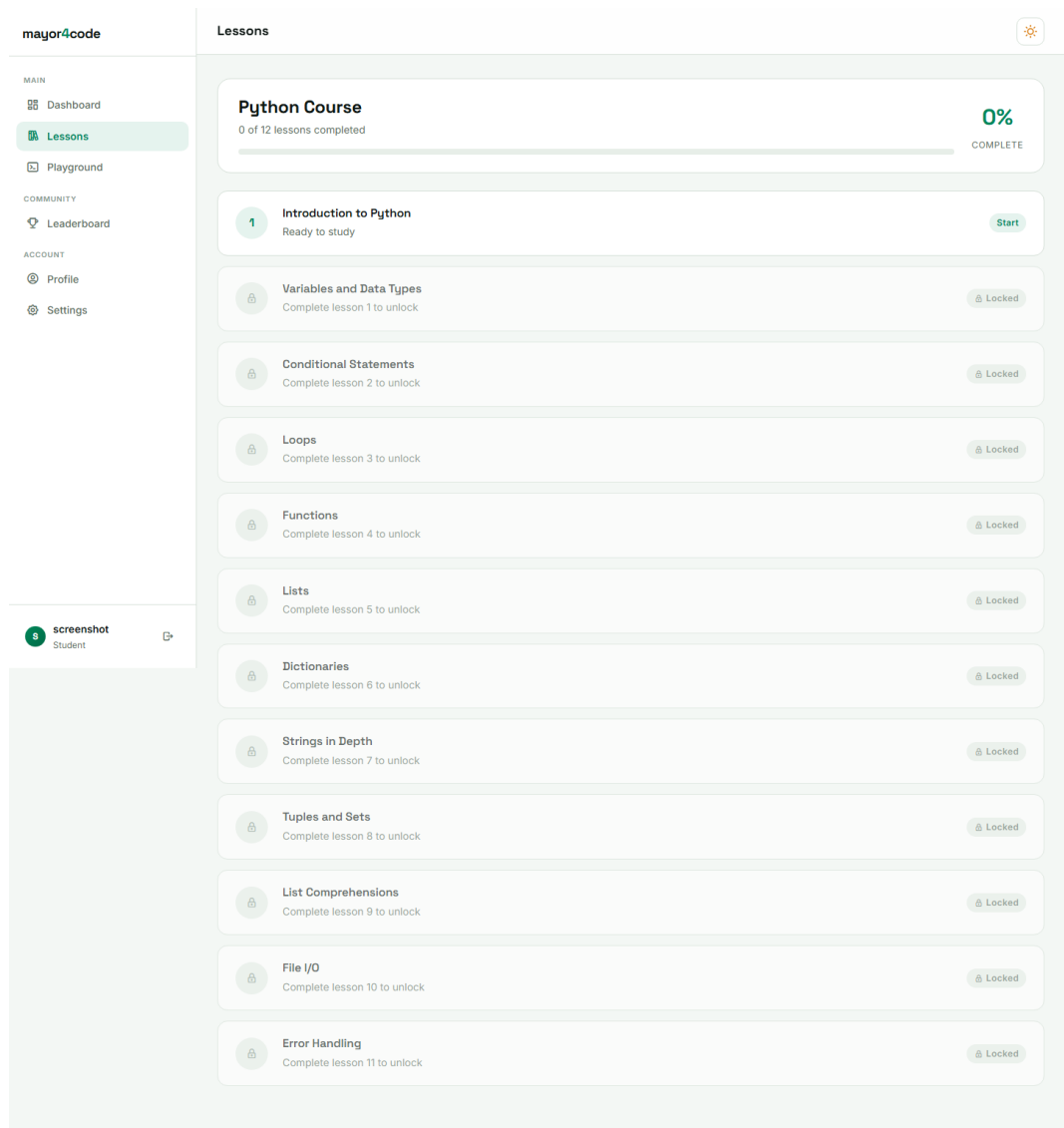


Figure 4.4: Lesson Interface of the mayor4code Platform

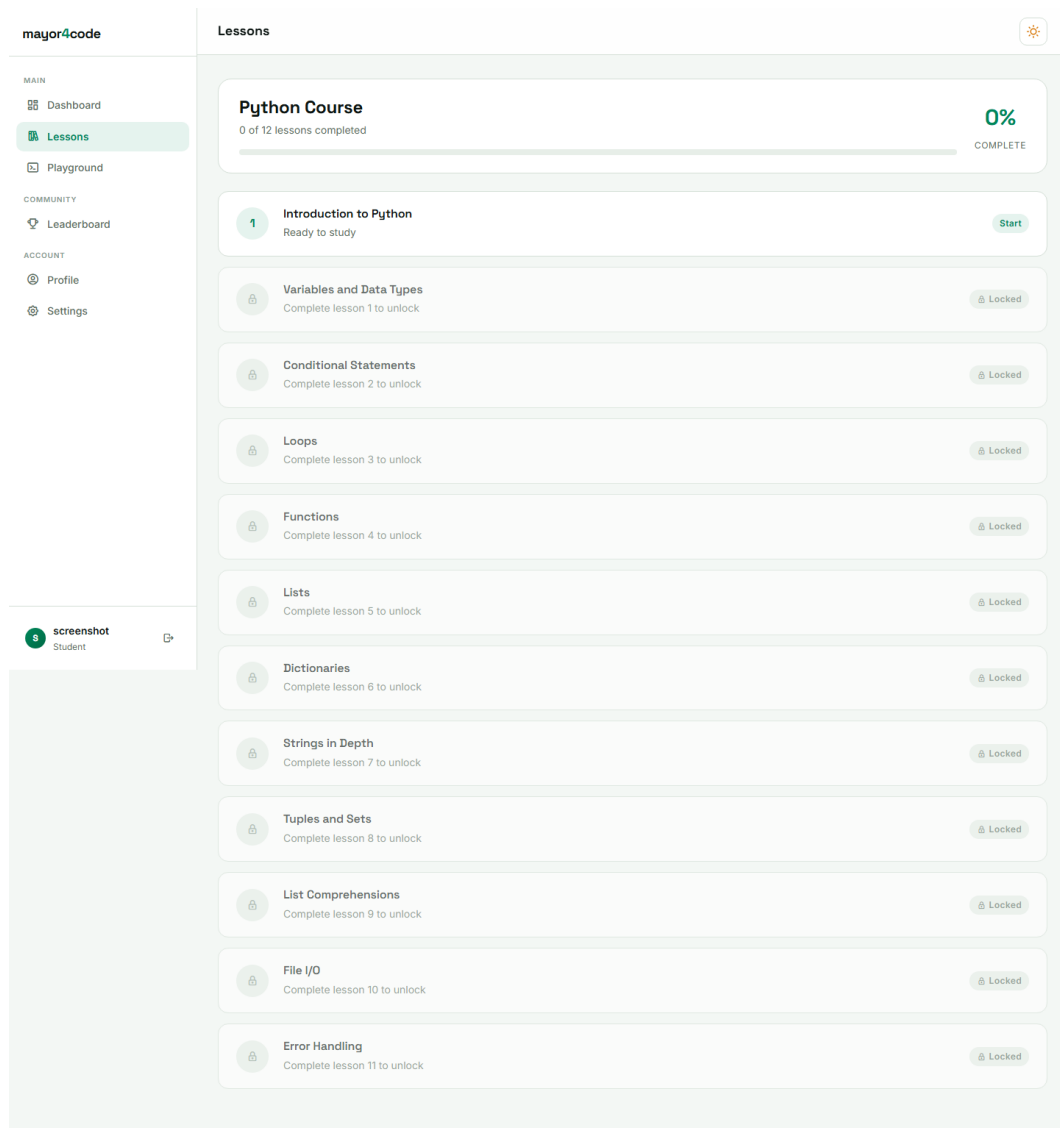


Figure 4.5: Quiz Interface of the mayor4code Platform

4.4.4 Python Playground Interface

The playground interface, shown in Figure 4.6, provides an in-browser editor where the learner writes Python code and executes it. The code is run on the server in an isolated subprocess with an enforced time limit, and the output is returned and displayed to the learner instantly.

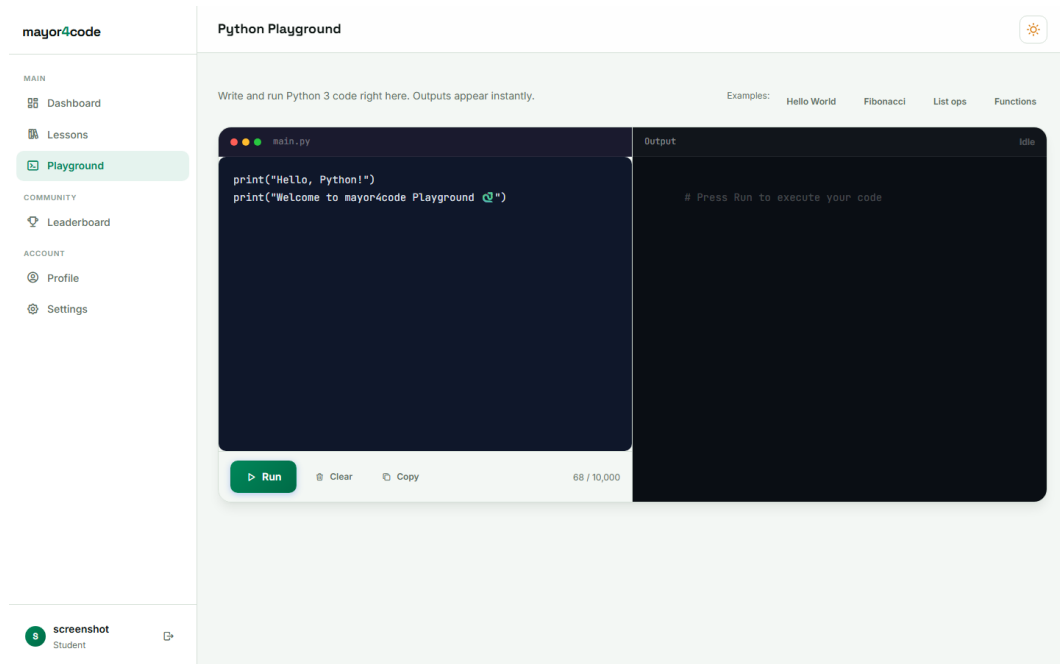


Figure 4.6: Python Playground Interface of the mayor4code Platform

4.4.5 Certificate and Leaderboard Interfaces

When a learner completes all lessons, the system automatically issues a completion certificate bearing a unique verification code, shown in Figure 4.7. The leaderboard interface, shown in Figure 4.8, ranks learners according to their average performance.

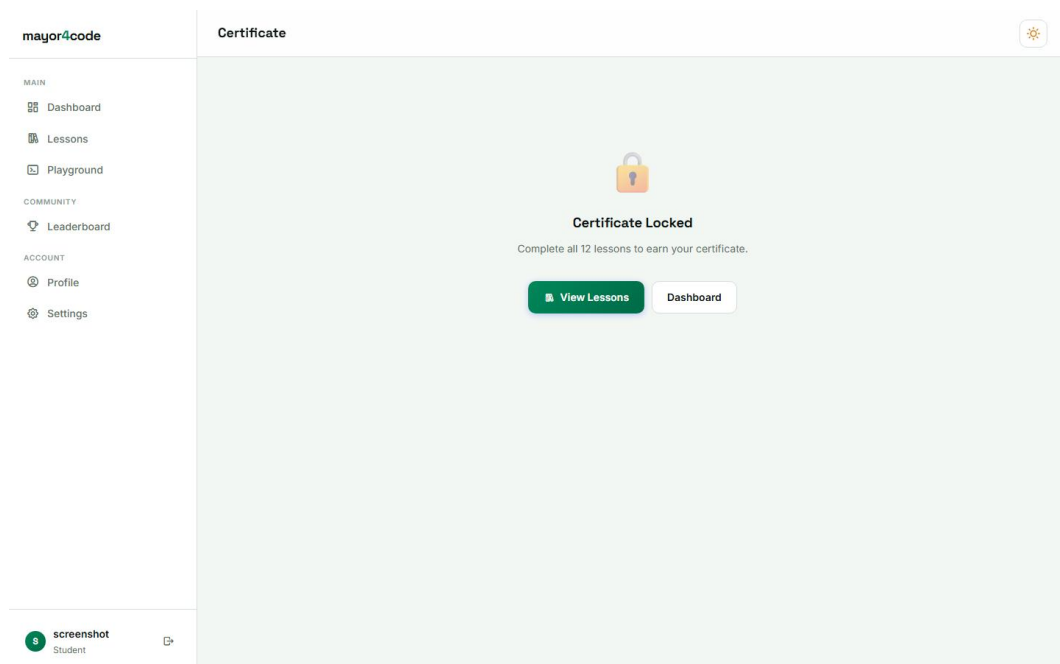


Figure 4.7: Completion Certificate of the mayor4code Platform

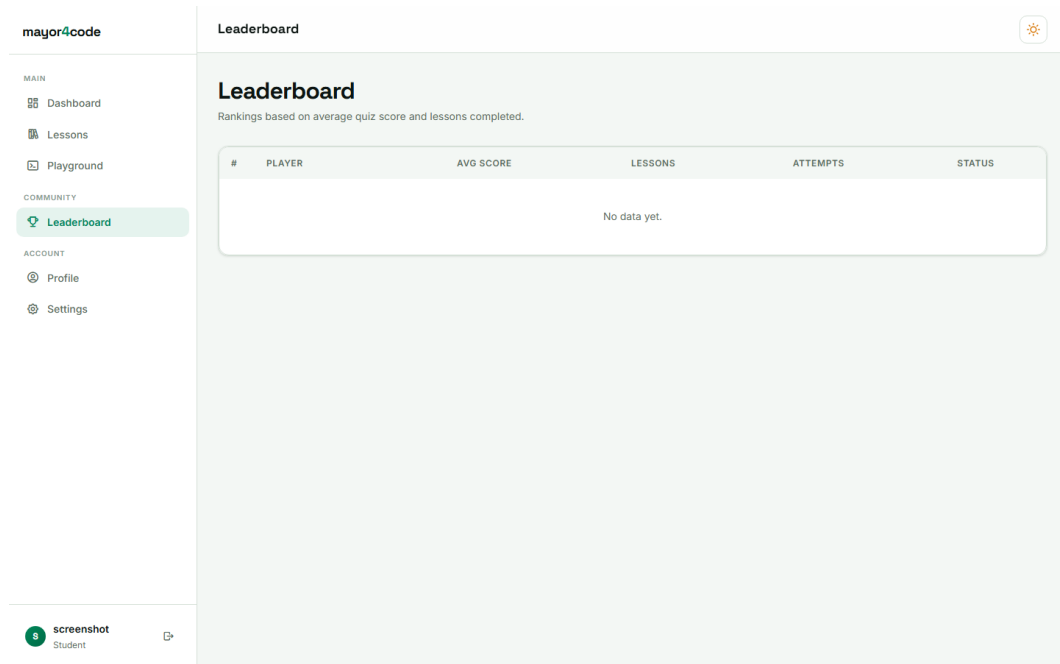


Figure 4.8: Leaderboard Interface of the mayor4code Platform

4.4.6 Administration Interface

The administration interface, shown in Figure 4.9, enables the administrator to manage lessons, quizzes, questions, users, and progress records, and to monitor overall system activity.

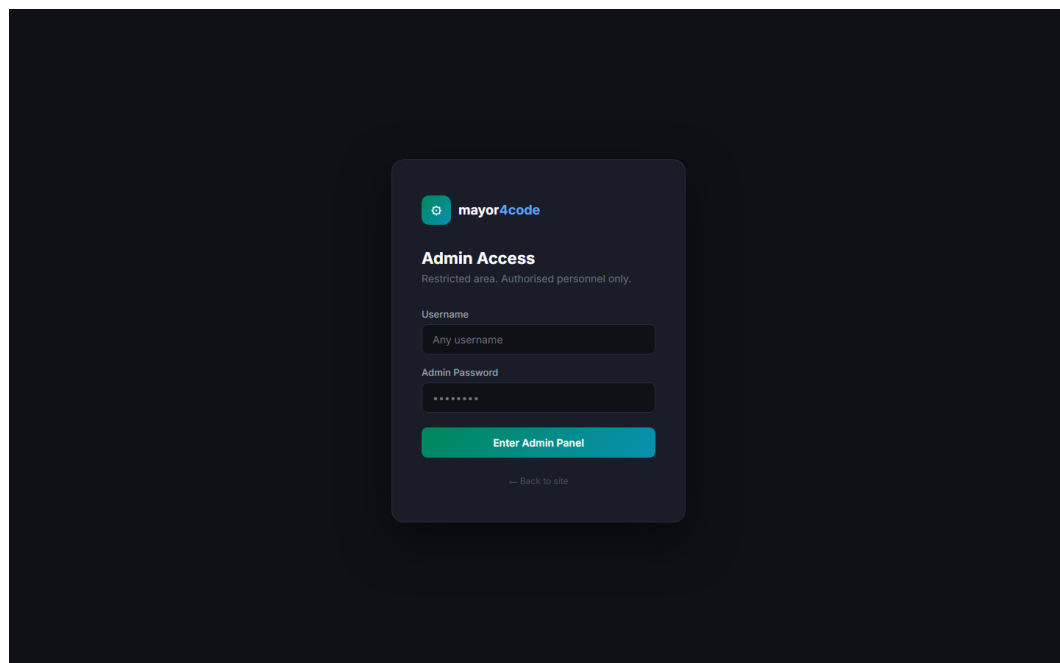


Figure 4.9: Administration Interface of the mayor4code Platform

4.5 System Testing

The system was tested to verify that it met its functional requirements and behaved reliably. Three levels of testing were carried out: unit testing of individual components, functional testing of complete features, and usability testing with sample users.

4.5.1 Unit Testing

Individual functions, such as quiz scoring, progression unlocking, and certificate-code generation, were tested in isolation to confirm that each produced the expected output for representative inputs.

4.5.2 Functional Testing

Complete features were tested end to end, including registration and login, lesson navigation, quiz submission and scoring, locked progression, code execution in the playground, certificate issuance, and leaderboard ranking. The results are summarised in Table 4.2.

Table 4.2: Summary of Test Cases and Results

- User registration and login — Accounts created and authenticated correctly. Passed.
- Locked progression — Next lesson unlocked only after a pass of at least sixty percent. Passed.
- Quiz scoring — Scores computed accurately and displayed instantly. Passed.
- Python playground — Valid code executed and returned output; unsafe operations and long-running code were prevented. Passed.
- Certificate issuance — Certificate with a unique code issued on course completion. Passed.
- Leaderboard — Learners ranked correctly by average performance. Passed.

4.5.3 Usability Testing

Sample users interacted with the platform and provided feedback on ease of use. Users found the interface intuitive and responsive, and the dark and light mode toggle was reported to improve reading comfort. Minor observations were addressed during iterative refinement.

4.6 Discussion of Results

The results confirmed that the platform met the objectives established in Chapter One. The system successfully delivered structured lessons with locked progression, provided an integrated in-browser coding environment, scored assessments automatically with instant feedback, tracked learner progress, and issued valid completion certificates alongside a motivational leaderboard.

In relation to the research objectives, each objective was realised: a structured, web-based lesson-delivery system was designed and implemented; a secure authentication mechanism was provided; an interactive Python playground was implemented; automated assessment with instant scoring was achieved; and progress tracking, certification, and a leaderboard were incorporated to enhance motivation. The functionality and usability of the platform were evaluated through testing.

In relation to the existing literature, the platform addresses the gaps identified in Chapter Two. Unlike several reviewed systems that restrict structured learning paths behind paywalls or fail to enforce mastery, mayor4code provides free, structured progression that enforces understanding of prerequisite concepts before advancement. By integrating lessons, an interactive playground, automated assessment, certification, and a leaderboard within a single coherent system, it also overcomes the fragmentation of tools observed in some existing platforms. These findings demonstrate the value of combining structured content, practical coding, and gamification in a single secure web application for introductory programming education.

CHAPTER FIVE SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary of Findings

This project set out to design and implement a web-based interactive e-learning platform for structured Python programming. The problem addressed was the difficulty beginners face in learning to program, arising from unstructured content, the separation of theory from practice, and the lack of immediate feedback, progress tracking, and recognition of achievement in many existing resources.

A review of relevant literature and existing platforms established the effectiveness of interactive, browser-based learning with automated feedback and gamification, while revealing gaps such as paywalls, weak enforcement of mastery, and the fragmentation of learning tools. Guided by these findings, the system was developed using the Agile approach and a three-tier architecture, implemented with the Django framework, Python, HTML, CSS, JavaScript, and a relational database.

The developed platform delivered twelve sequential Python lessons with locked progression, multiple-choice quizzes with instant scoring, an in-browser Python playground with safe code execution, progress tracking, automatically issued completion certificates, and a leaderboard. Testing confirmed that the system enforced progression correctly, scored assessments accurately, executed code safely, and issued valid certificates.

5.2 Conclusion

The study successfully achieved its aim of designing and implementing an integrated, web-based interactive e-learning platform for structured Python programming. By combining structured lesson delivery, mastery-based progression, practical in-browser coding, automated assessment, certification, and gamification within a single secure application, the platform improves the accessibility and effectiveness of introductory programming education. The system therefore provides a practical solution to the challenges that beginners commonly encounter when learning to program.

5.3 Recommendations

Based on the outcomes of this study, the following recommendations are made:

12. Educational institutions and training organisations should adopt integrated, interactive e-learning platforms of this kind to complement traditional programming instruction.
13. The principle of mastery-based locked progression should be applied in programming education to reduce knowledge gaps among learners.
14. Adequate attention should be given to the security of in-browser code-execution environments, including strong isolation of executed code, to protect the hosting infrastructure.
15. Learner motivation should be supported through gamification features such as certification and leaderboards.

5.4 Suggestions for Future Research

The following suggestions are offered for future work:

16. Extending the platform to cover additional programming languages and more advanced topics beyond introductory Python.
17. Incorporating adaptive learning that personalises content and pacing based on each learner's performance.
18. Strengthening the code-execution environment through container-based sandboxing with stricter resource, network, and filesystem controls.
19. Adding richer assessment types, such as automatically graded coding exercises, in addition to multiple-choice quizzes.
20. Conducting a large-scale empirical evaluation of learning outcomes to measure the platform's educational impact quantitatively.

REFERENCES

- Anderson, T. (2008). *The theory and practice of online learning* (2nd ed.). Athabasca University Press.
- Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining “gamification”. In *Proceedings of the 15th International Academic MindTrek Conference* (pp. 9–15). Association for Computing Machinery.
- Django Software Foundation. (2024). Django documentation. <https://docs.djangoproject.com/>
- Duckett, J. (2014). *HTML and CSS: Design and build websites*. John Wiley & Sons.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7th ed.). Pearson.
- Garrison, D. R. (2017). *E-learning in the 21st century: A community of inquiry framework for research and practice* (3rd ed.). Routledge.
- Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences* (pp. 3025–3034). IEEE.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner’s approach* (9th ed.). McGraw-Hill Education.
- Python Software Foundation. (2024). The Python language reference. <https://docs.python.org/3/reference/>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.